



Microcontrollers

Rex Joseph

Department of Electrical and
Electronics Engineering

MSP430 Assembly Language Programming

Basic assembly language syntax

An assembly language source program is a **text file** made up of a series of **source statements**.

A source statement will contain **one or more** of the following **elements**:

Label - acts as a **marker** in the source code, useful as the destination in branching statements and subroutine calls

Mnemonic - a **machine instruction, assembler directive, or macro**

Operand List - one or more values to be operated on by the instruction, directive, or macro

Comment - note inserted by the programmer that is not included in the assembly

<http://robotics.hobbizine.com/asmlau.html> , SLAU131Y – OCTOBER 2004 – REVISED JUNE 2021

MSP430 Assembly Language Programming

Example of source statement

label	mnemonic	operand list	comment
Mainloop	bit.b	#00001000b,&P1IN	; Read switch at P1.3

Assembler directives can be distinguished from machine instructions by their leading dot, for example: (not in IaR)

.end is the directive which tells the assembler that it has reached the last source statement of a program.

Labels are optional.

When used they **must be the first character of a line.**

They cannot begin with a number or contain a blank space, but otherwise can include alphanumeric characters A-Z, a-z, 0-9, _, and \$.

MSP430 Assembly Language Programming

Labels are case sensitive by default but can be set as case insensitive as an option.

A label can be followed by a colon(:), but this is not required and the colon is not treated as part of the label name when referenced elsewhere in the program.

Comments are also optional. They can use any ASCII character including spaces. When a comment starts at the first character in a line it can begin with either an asterisk(*) or a semicolon(;), if it does not begin at the first character it can only begin with a semicolon.

The first character of a line can only be a label, asterisk, semicolon, or a blank space.

The **first character cannot be a mnemonic** or it will be treated as a label by the assembler.

MSP430 Assembly Language Programming

The MSP430 assembler reads an unlimited number of characters per line. Source statements that extend beyond 400 characters in length (including comments) are truncated in the listing file.

Follow these guidelines:

- All statements must begin with a label, a blank, an asterisk, or a semicolon.
- Labels are optional for most statements; if used, they must begin in column 1.
- One or more space or tab characters must separate each field.
- Comments are optional. Comments that begin in column 1 can begin with an asterisk or a semicolon (* or ;), but comments that begin in any other column must begin with a semicolon.

MSP430 Assembly Language Programming

Note

A mnemonic cannot begin in column 1 or it will be interpreted as a label.

Mnemonic opcodes and assembler directive names without the . prefix **are valid label names.**

Remember to always use whitespace before the mnemonic, or the assembler will think the identifier is a new label definition.

SLAU131Y – OCTOBER 2004 – REVISED JUNE 2021

Debugging

```

18 ;-----
19 RESET      mov.w   #__STACK_END,SP      ; Initialize stackpointer
20 StopWDT    mov.w   #WDTPW|WDTHOLD,&WDTCTL ; Stop watchdog timer
21

```

Name	Value
Core Registers	
PC	0xC000
SP	0x03FE
SR	0x0000
R3	0x0000
R4	0x24EB

```

18 ;-----
19 RESET      mov.w   #__STACK_END,SP      ; Initialize stackpointer
20 StopWDT    mov.w   #WDTPW|WDTHOLD,&WDTCTL ; Stop watchdog timer
21

```

Core Registers	
PC	0xC004
SP	0x0400
SR	0x0000
R3	0x0000
R4	0x0000

```

;-----
RESET      mov.w   #__STACK_END,SP
           mov.w   #0x0120,R6
           mov.w   #0x5A80,0(R6)
;-----

```

```

0xBFDA 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF
0xC000 $../main.asm:19:34$, RESET
0xC000 4031 0400
0xC004 StopWDT
0xC004 40B2 5A80 0120
0xC00A Mainloop
0xC00A 403F 072A 407E 072A 3FFB
0xC014 $isr_trap.asm:48:59$, __TI_ISR_TRAP
0xC014 D032 0010 3FFD 4303 FFFF FFFF FFFF FFFF

```

Debugging

```

26
27 Mainloop  mov.w  #1834,R15
28          mov.b  #1834,R14
29          jmp    Mainloop
30

```

Core Registers	
PC	0xC00A
SP	0x0400
SR	0x0000
R14	0x002C
R15	0x0E50

```

26
27 Mainloop  mov.w  #1834,R15
28          mov.b  #1834,R14
29          jmp    Mainloop
30

```

PC	0xC00E
SP	0x0400
R13	0xB67F
R14	0x002C
R15	0x072A

```

0xBFDA  3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF
0xC000  $../main.asm:19:34$, RESET
0xC000  4031 0400
0xC004  StopWDT
0xC004  40B2 5A80 0120
0xC00A  Mainloop
0xC00A  403F 072A 407E 072A 3FFB
0xC014  $isr_trap.asm:48:59$, __TI_ISR_TRAP
0xC014  D032 0010 3FFD 4303 FFFF FFFF FFFF FFFF

```

What will R14 be after instruction is executed?

Debugging

```
--
27 Mainloop    mov.w  #1834,R15
28             mov.b  #1834,R14
29             jmp    Mainloop
30 ;
```

1010 0101	PC	0xC012
1010 0101	SP	0x0400
1010 0101	R13	0xB67F
1010 0101	R14	0x002A
1010 0101	R15	0x072A

Why not 0x072A??
What comes next??

```
0xBFDA  3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF
0xC000  $../main.asm:19:34$, RESET
0xC000  4031 0400
0xC004  StopWDT
0xC004  40B2 5A80 0120
0xC00A  Mainloop
0xC00A  403F 072A 407E 072A 3FFB
0xC014  $isr_trap.asm:48:59$, __TI_ISR_TRAP
0xC014  D032 0010 3FFD 4303 FFFF FFFF FFFF FFFF FFFF
```

Debugging

```
--
27 Mainloop    mov.w  #1834,R15
28             mov.b  #1834,R14
29             jmp    Mainloop
30 ;
```

1010 0101	PC	0xC012
1010 0101	SP	0x0400
1010 0101	R13	0xB67F
1010 0101	R14	0x002A
1010 0101	R15	0x072A

```
26
27 Mainloop    mov.w  #1834,R15
28             mov.b  #1834,R14
29             jmp    Mainloop
30 ;
```

1010 0101	PC	0xC00A
1010 0101	SP	0x0400
1010 0101	R13	0xB67F

Why not 0x072A??

What comes next??

Where is memory 0xC012? Instruction??

```
0xBFDA  3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF
0xC000  ../main.asm:19:34$, RESET
0xC000  4031 0400
0xC004  StopWDT
0xC004  40B2 5A80 0120
0xC00A  Mainloop
0xC00A  403F 072A 407E 072A 3FFB
0xC014  $isr_trap.asm:48:59$, __TI_ISR_TRAP
0xC014  D032 0010 3FFD 4303 FFFF FFFF FFFF FFFF FFFF
```

Debugging

The source and destination of an instruction are defined by the following fields:

src	The source operand defined by As and S-reg
dst	The destination operand defined by Ad and D-reg
As	The addressing bits responsible for the addressing mode used for the source (src)
S-reg	The working register used for the source (src)
Ad	The addressing bits responsible for the addressing mode used for the destination (dst)
D-reg	The working register used for the destination (dst)
B/W	Byte or word operation: 0: word operation 1: byte operation

Table 4-3. Source/Destination Addressing

As/Ad	Addressing Mode	Syntax	Description
00/0	Register	Rn	Register contents are operand.
01/1	Indexed	X(Rn)	(Rn + X) points to the operand. X is stored in the next word, or stored in combination of the preceding extension word and the next word.
01/1	Symbolic	ADDR	(PC + X) points to the operand. X is stored in the next word, or stored in combination of the preceding extension word and the next word. Indexed mode X(PC) is used.
01/1	Absolute	&ADDR	The word following the instruction contains the absolute address. X is stored in the next word, or stored in combination of the preceding extension word and the next word. Indexed mode X(SR) is used.
10/–	Indirect Register	@Rn	Rn is used as a pointer to the operand.
11/–	Indirect Autoincrement	@Rn+	Rn is used as a pointer to the operand. Rn is incremented afterwards by 1 for .B instructions, by 2 for .W instructions, and by 4 for .A instructions.
11/–	Immediate	#N	N is stored in the next word, or stored in combination of the preceding extension word and the next word. Indirect autoincrement mode @PC+ is used.

Debugging

0xC004 403F 072A 407E 072A 3FFB

Mainloop mov.w #1834,R15 ; MOV operation, word, immediate
 mov.b #1834,R14
 jmp Mainloop

Figure 3-9. Double Operand Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code				S-Reg				Ad	B/W	As		D-Reg			

Table 3-11 lists and describes the double operand instructions.

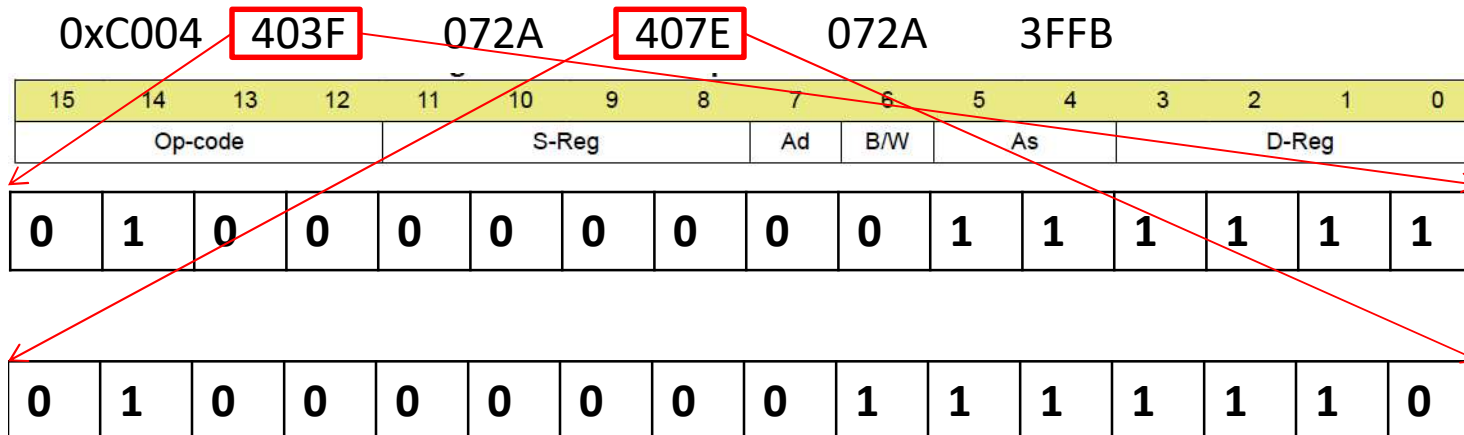
Table 3-11. Double Operand Instructions

Mnemonic	S-Reg, D-Reg	Operation	Status Bits			
			V	N	Z	C
MOV (.B)	src, dst	src → dst	-	-	-	-

11/-	Immediate mode	#N	The word following the instruction contains the immediate constant N. Indirect autoincrement mode @PC+ is used.
------	----------------	----	---

Addressing Modes - Immediate

Mainloop mov.w #1834,R15 ; MOV operation, word, immediate
 mov.b #1834,R14
 jmp Mainloop



Addressing Modes - Register

Mainloop **mov.w** **#1834,R15 ; MOV operation, word, immediate**
 mov.b **#1834,R14**
 mov.w **R15,R14**
 jmp **Mainloop**

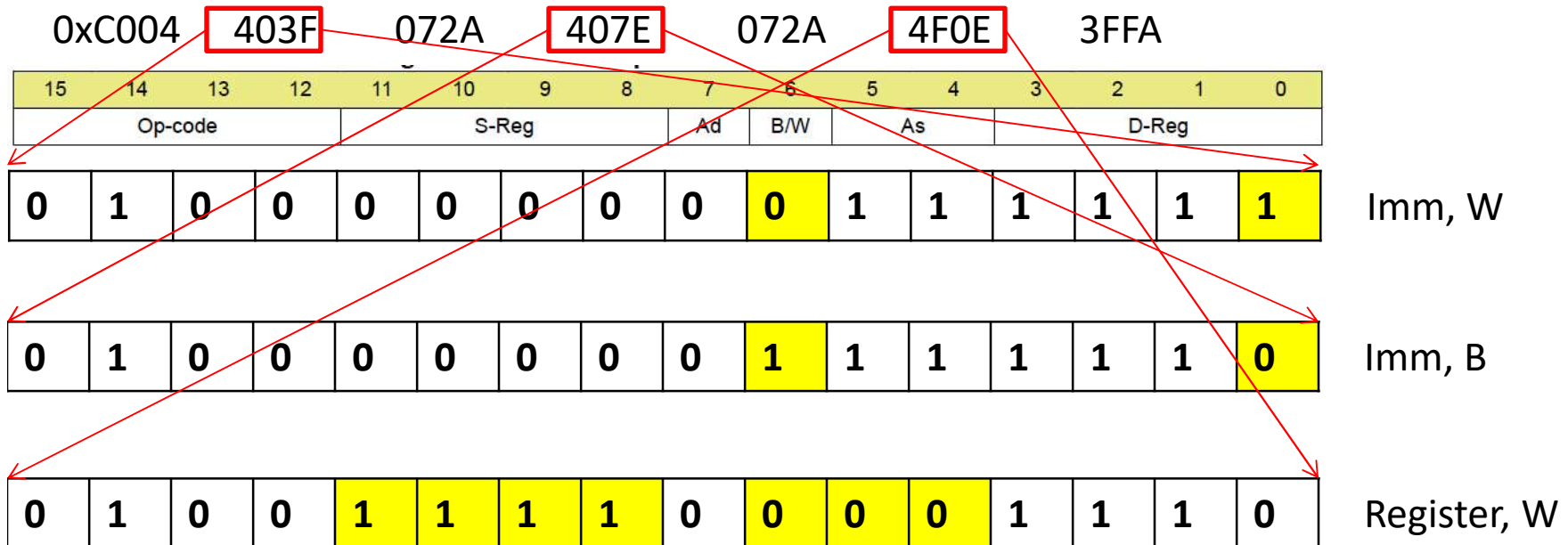
1010 0101	R13	0xB67F
1010 0101	R14	0x072A
1010 0101	R15	0x072A

```
--
?7 Mainloop  mov.w  #1834,R15
?8           mov.b  #1834,R14
?9           mov.w  R15,R14
?0           jmp    Mainloop
?1 ;-----
```

```
0xBFDA 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF 3FFF
0xC000 $../main.asm:19:35$, RESET
0xC000 4031 0400
0xC004 StopWDT
0xC004 40B2 5A80 0120
0xC00A Mainloop
0xC00A 403F 072A 407E 072A 4F0E 3FFA
0xC016 $isr_trap.asm:48:59$, __TI_ISR_TRAP
0xC016 |D032 0010 3FFD 4303 FFFF FFFF FFFF FFFF
```

Addressing Modes - Register

Mainloop mov.w #1834,R15 ; MOV operation, word, immediate
 mov.b #1834,R14
 jmp Mainloop



Addressing Modes - Indexed

26				0xC000	\$/./main.asm:19:36\$, RESET
27	Mainloop	mov.w	#1834,R15	0xC000	4031 0400
28		mov.b	#1834,R14	0xC004	StopWDT
29		mov.w	R15,R14	0xC004	40B2 5A80 0120
30		ADD.B	1000h(R5),0F000h(R6)	0xC00A	Mainloop
31		jmp	Mainloop	0xC00A	403F 072A 407E 072A 4F0E 55D6 1000 F000 3FF7
32				0xC01C	\$/isr_trap.asm:48:59\$, __TI_ISR_TRAP
				0xC01C	D032 0010 3FFD 4303 FFFF FFFF FFFF FFFF FFFF FFFF

Core Registers		
1010 0101	PC	0xC01A
1010 0101	SP	0x0400
1010 0101	SR	0x0005
1010 0101	R3	0x0000
1010 0101	R4	0x24EB
1010 0101	R5	0x497C
1010 0101	R6	0x1778
1010 0101	R7	0xFCF5
1010 0101	R8	0x7FF7
1010 0101	R9	0x4C11
1010 0101	R10	0x0000
1010 0101	R11	0xFBFD
1010 0101	R12	0x5FFF
1010 0101	R13	0xB67F
1010 0101	R14	0x072A
1010 0101	R15	0x072A

0x0752	3FFF 3FFF 3FFF 3FFF 3FFF 3FFF :
0x0778	3FFF 3FFF 3FFF 3FFF 3FFF 3FFF :
0x079E	3FFF 3FFF 3FFF 3FFF 3FFF 3FFF :
0x07C4	3FFF 3FFF 3FFF 3FFF 3FFF 3FFF :

Operation: The signed **16-bit index** is located in the **next word after the instruction** and is added to the CPU register Rn.

The resulting bits 19:16 are cleared giving a truncated 16-bit memory address, which points to an operand address in the range 00000h to 0FFFFh.

The operand is the content of the addressed memory location

Source: The byte pointed to by R5 + 1000h = 0x579Ch after truncation

Destination: The byte pointed to by R6 + F000h = 00778h after truncation

This instruction adds the 8-bit data contained in source byte 1000h(R5) and the destination byte 0F000h(R6) and places the result into the destination byte

Instruction set (part)

ADC

*ADC[.W] Add carry to destination ; *ADC.B Add carry to destination

Syntax: ADC dst or ADC.W dst; ADC.B dst

Operation dst + C → dst

Emulation ADDC #0,dst; ADDC.B #0,dst

Description The carry bit (C) is added to the destination operand. The previous content of the destination is lost.

Status Bit N: Set if result is negative, reset if positive

Z: Set if result is zero, reset otherwise

C: Set if dst was incremented from 0FFFFh to 0000, reset otherwise
Set if dst was incremented from 0FFh to 00, reset otherwise

V: Set if an arithmetic overflow occurs, otherwise reset

Mode Bits OSCOFF, CPUOFF, and GIE are not affected.

ADD @R13,0(R12) ; Add LSDs

ADC 2(R12) ; Add carry to MSD

Instruction set (part)

ADD

ADD[.W]; ADD.B Add source to destination

Syntax ADD src,dst or ADD.W src,dst; ADD.B src,dst

Operation src + dst → dst

Description The source operand is added to the destination operand.
The source operand is not affected. The previous contents
of the destination are lost.

Status Bits N: Set if result is negative, reset if positive
Z: Set if result is zero, reset otherwise
C: Set if there is a carry from the result, cleared if not
V: Set if an arithmetic overflow occurs, otherwise reset

Mode Bits OSCOFF, CPUOFF, and GIE are not affected

ADD #10,R5

JC TONI ; Carry occurred

..... ; No carry

Instruction set (part)

BIT

BIT[.W]; BIT.B Test bits in destination

Syntax BIT src,dst or BIT.W src,dst

Operation src .AND. dst

Description The source and destination operands are logically ANDed.
The result affects only the status bits. The source and destination operands are not affected.

Status Bits
N: Set if MSB of result is set, reset otherwise
Z: Set if result is zero, reset otherwise
C: Set if result is not zero, reset otherwise (.NOT. Zero)
V: Reset

Mode Bits OSCOFF, CPUOFF, and GIE are not affected.

Example If bit 9 of R8 is set, a branch is taken to label TOM.

```

    BIT #0200h,R8      ; bit 9 of R8 set?
    JNZ TOM            ; Yes, branch to TOM
    ...                ; No, proceed
  
```

Instruction set (part)

*BR, BRANCH Branch to destination

Syntax BR dst

Operation dst → PC

Emulation MOV dst,PC

Description An unconditional branch is taken to an address anywhere in the 64K address space. All source addressing modes can be used. The branch instruction is a word instruction.

Status Bits Status bits are not affected.

Instruction set (part)

CMP

CMP[.W]: CMP.B Compare source and destination

Syntax CMP src,dst or CMP.W src,dst; CMP.B src,dst

Operation dst + .NOT.src + 1 or (dst - src)

Description The source operand is subtracted from the destination operand. This is accomplished by adding the 1s complement of the source operand plus 1. The two operands are not affected and the result is not stored; only the status bits are affected.

Status Bits N: Set if result is negative (src > dst), reset if positive (src ≤ dst)
 Z: Set if result is zero (src = dst), reset otherwise (src ≠ dst)
 C: Set if there is a carry from the MSB of the result, reset otherwise
 V: Set if an arithmetic overflow occurs, otherwise reset

Mode Bits OSCOFF, CPUOFF, and GIE are not affected



THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 515